

E-Commerce – Sprint 1

System Architecture & Scope

Project Title: Mobile Store E-Commerce System

Course: E-Commerce

Sprint: 1 – System Architecture & Scope

Documentation File: "SPRINT_1.md"

1. Target Audience & Market Focus

1.1 Primary Persona

The primary users of the Mobile Store E-Commerce System are retail consumers looking to purchase smartphones online. The platform is designed for users who want to browse different mobile phones, compare their specifications and prices, read reviews from other customers, and purchase a suitable device through an easy-to-use online store.

The system may also support an administrator role responsible for managing products, categories, inventory, orders, and customer reviews.

1.2 Core Pain Point

Customers often have difficulty choosing the right smartphone because there are many models available with different specifications, prices, features, and user experiences. Customers may need to visit multiple websites to compare mobile phones and find reliable opinions from existing users.

The proposed system addresses this problem by providing a centralized mobile shopping platform where users can:

- Browse and search smartphones.
- Filter products by relevant categories and specifications.
- Compare multiple mobile phones side-by-side.
- Read customer ratings and reviews.
- Add products to a shopping cart.
- Place orders through an online checkout process.
- Make informed purchasing decisions based on specifications, prices, and user experiences.

1.3 Domain Scope

The system focuses specifically on the Consumer Electronics – Smartphones/Mobile Phones market.

The platform will provide an online marketplace/store for smartphones from different brands. Each product will contain information such as its name, brand, price, specifications, available stock, and category.

The system will also incorporate customer reviews and ratings for products and a mobile comparison feature that allows customers to compare important specifications and prices of selected smartphones.

2. Minimum Viable Product (MVP) Feature Scope

The Minimum Viable Product will focus on the core workflows required to operate a functional online mobile store. The selected features are designed to remain feasible within the academic semester while providing the major functionality expected from the platform.

Category	Feature	Description	Priority
Authentication	User Registration & Authentication	Users can create accounts and securely log in using password hashing and JWT-based authentication.	High (MVP)
Catalog	Mobile Product Listing & Search	Users can browse smartphones and search for products by name, brand, or other relevant information.	High (MVP)
Catalog	Product Filtering	Users can filter mobile phones based on category, brand, price range, and selected specifications.	High (MVP)
Product	Product Details	Users can view detailed information including price, specifications, stock availability, and product information.	High (MVP)
Comparison	Mobile Comparison	Users can select multiple smartphones and compare their specifications, prices, and other important features side-by-side.	High (MVP)
Cart	Cart Management	Users can add smartphones to their cart, update quantities, and remove products before checkout.	High (MVP)

2.1 Core User Workflow

The primary customer workflow will be:

```

Register / Login
↓
Browse Mobile Phones
↓
Search / Filter
↓
View Product Details
↓
Read Customer Reviews
↓
Compare Mobile Phones
↓
Add Product to Cart
↓
Checkout
↓
Place Order

```

This workflow represents the main functionality of the Mobile Store system.

3. Tech Stack Selection & Justification

3.1 Frontend Framework – React.js

Selected Technology: React.js

React.js will be used to develop the frontend user interface of the Mobile Store. It provides a component-based architecture that is suitable for building reusable elements such as product cards, search interfaces, comparison tables, review sections, shopping carts, and checkout pages.

React was selected because it provides a flexible and well-established ecosystem for interactive e-commerce interfaces. Compared with traditional server-rendered frontend approaches, React makes it easier to create dynamic interfaces such as real-time product filtering and mobile comparison.

3.2 Backend Infrastructure – Node.js / Express.js

Selected Technology: Node.js with Express.js

Node.js will be used as the server-side runtime environment, while Express.js will be used as the backend web framework for developing REST APIs and implementing the business logic of the application. The backend will handle authentication, product management, reviews, mobile comparison, cart operations, order processing, and communication with the PostgreSQL database.

Node.js and Express.js were selected because they provide a lightweight, scalable, and efficient environment for developing RESTful APIs. Using JavaScript on both the frontend and backend also simplifies development by allowing the project team to work with a consistent programming language across the application. Compared with larger backend frameworks, Express.js provides a flexible architecture that is suitable for an academic e-commerce project.

3.3 Database Management System – PostgreSQL

Selected Technology: PostgreSQL

PostgreSQL will be used as the primary relational database management system. It will store structured information about users, mobile products, categories, reviews, carts, orders, and order items.

PostgreSQL is suitable because the Mobile Store requires strong relationships and data integrity between multiple entities. A relational database is preferable to a non-relational database for this project because entities such as users, products, orders, and reviews have clearly defined relationships and require consistent data.

3.4 Caching & Asynchronous Processing – Redis (Optional)

Selected Technology: Redis

Redis may optionally be used for caching frequently accessed information and improving application performance. It can also be used for temporary session-related data or other short-lived application data.

Redis is not considered a mandatory MVP component and may be introduced if performance or caching requirements arise during development.

4. Entity-Relationship Diagram (ERD)

The database will follow a relational model containing the major entities required by the Mobile Store system.

4.1 Main Entities

The proposed database contains the following entities:

1. Users
2. Categories
3. Products
4. Reviews
5. Orders
6. Order_Items
7. Cart
8. Cart_Items

Additional product attributes can be represented through the Products entity or expanded into separate entities if required during later development.

4.2 Entity Descriptions

Users

Stores customer and administrator account information.

Important attributes:

- "id" – Primary Key
- "name"
- "email"
- "password_hash"

- "role"
- "created_at"

Categories

Stores mobile product categories.

Important attributes:

- "id" – Primary Key
- "name"
- "description"

Products

Stores smartphone information.

Important attributes:

- "id" – Primary Key
- "category_id" – Foreign Key
- "name"
- "brand"
- "description"
- "price"
- "stock_quantity"
- "ram"
- "storage"
- "display"

- "camera"
- "battery"
- "processor"
- "created_at"

These attributes provide the information required for product browsing and mobile comparison.

Reviews

Stores customer ratings and written reviews for smartphones.

Important attributes:

- "id" – Primary Key
- "user_id" – Foreign Key
- "product_id" – Foreign Key
- "rating"
- "comment"
- "created_at"

A customer can submit a review for a product, and each product can have multiple customer reviews.

Orders

Stores information about customer orders.

Important attributes:

- "id" – Primary Key

- "user_id" – Foreign Key
- "total_amount"
- "status"
- "created_at"

Order_Items

Associative entity connecting orders with products.

Important attributes:

- "id" – Primary Key
- "order_id" – Foreign Key
- "product_id" – Foreign Key
- "quantity"
- "unit_price"

This entity allows one order to contain multiple products and allows a product to appear in multiple orders.

Cart

Stores the shopping cart associated with a user.

Important attributes:

- "id" – Primary Key
- "user_id" – Foreign Key
- "created_at"

- "updated_at"

Cart_Items

Stores the individual products contained in a shopping cart.

Important attributes:

- "id" – Primary Key
- "cart_id" – Foreign Key
- "product_id" – Foreign Key
- "quantity"

4.3 Relationships & Cardinality

The major relationships are:

Users → Orders

One user can place many orders.

Relationship: "1:N"

One User → Many Orders

Orders → Order_Items

One order can contain multiple order items.

Relationship: "1:N"

One Order → Many Order_Items

Products → Order_Items

One product can appear in many order items.

Relationship: "1:N"

One Product → Many Order_Items

Categories → Products

One category can contain multiple products.

Relationship: "1:N"

One Category → Many Products

Users → Reviews

One user can write multiple reviews.

Relationship: "1:N"

One User → Many Reviews

Products → Reviews

One product can receive multiple customer reviews.

Relationship: "1:N"

One Product → Many Reviews

Users → Cart

A user has one active shopping cart.

Relationship: "1:1"

One User → One Cart

Cart → Cart_Items

One cart can contain multiple cart items.

Relationship: "1:N"

One Cart → Many Cart_Items

Products → Cart_Items

One product can appear in multiple users' carts.

Relationship: "1:N"

One Product → Many Cart_Items

4.4 Mermaid ERD

The following Mermaid diagram can be embedded directly into "SPRINT_1.md".

erDiagram

USERS ||--o{ ORDERS : places

USERS ||--o{ REVIEWS : writes

USERS ||--|| CART : owns

CATEGORIES ||--o{ PRODUCTS : contains

PRODUCTS ||--o{ REVIEWS : receives

PRODUCTS ||--o{ ORDER_ITEMS : included_in

PRODUCTS ||--o{ CART_ITEMS : added_to

ORDERS ||--||{ ORDER_ITEMS : contains

CART ||--o{ CART_ITEMS : contains

USERS {

INTEGER id PK

```
    VARCHAR name
    VARCHAR email
    VARCHAR password_hash
    VARCHAR role
    TIMESTAMP created_at
}
```

```
CATEGORIES {
    INTEGER id PK
    VARCHAR name
    VARCHAR description
}
```

```
PRODUCTS {
    INTEGER id PK
    INTEGER category_id FK
    VARCHAR name
    VARCHAR brand
    TEXT description
    DECIMAL price
    INTEGER stock_quantity
    VARCHAR ram
    VARCHAR storage
    VARCHAR display
    VARCHAR camera
    VARCHAR battery
    VARCHAR processor
    TIMESTAMP created_at
}
```

REVIEWS {

INTEGER id PK

INTEGER user_id FK

INTEGER product_id FK

INTEGER rating

TEXT comment

TIMESTAMP created_at

}

ORDERS {

INTEGER id PK

INTEGER user_id FK

DECIMAL total_amount

VARCHAR status

TIMESTAMP created_at

}

ORDER_ITEMS {

INTEGER id PK

INTEGER order_id FK

INTEGER product_id FK

INTEGER quantity

DECIMAL unit_price

}

CART {

INTEGER id PK

INTEGER user_id FK

```
    TIMESTAMP created_at
    TIMESTAMP updated_at
}
```

```
CART_ITEMS {
    INTEGER id PK
    INTEGER cart_id FK
    INTEGER product_id FK
    INTEGER quantity
}
```

5. Mobile Comparison Feature

The Mobile Comparison feature is one of the key differentiating features of the system.

Users will be able to select multiple smartphones and compare their important attributes in a structured comparison interface.

For example:

Specification| Mobile A| Mobile B| Mobile C

Brand| Brand A| Brand B| Brand C

Price| Price| Price| Price

RAM| 8 GB| 12 GB| 8 GB

Storage| 256 GB| 256 GB| 128 GB

Display| Specification| Specification| Specification

Camera| Specification| Specification| Specification

Battery| Specification| Specification| Specification

Processor| Specification| Specification| Specification

Customer Rating| Rating| Rating| Rating

The comparison feature will help customers make better purchasing decisions without manually switching between different product pages.

6. Customer Reviews & Ratings

Customer reviews will provide real user feedback for individual smartphones.

A customer who has purchased or is eligible to review a product can provide:

- A numerical rating.
- A written review.
- The date of the review.

Example:

Product: Smartphone A

★★★★★

"Excellent performance and good battery life."

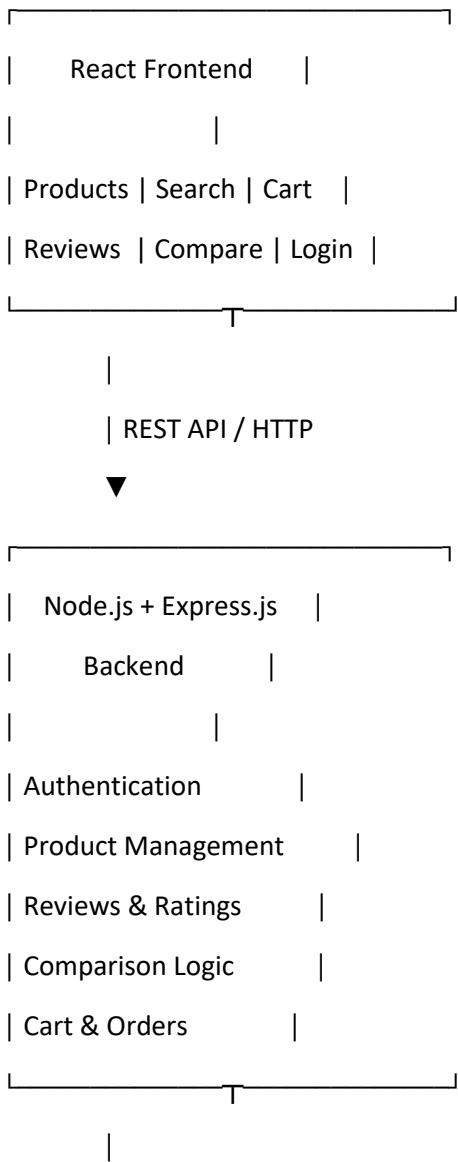
— Customer

The product page will display the overall customer rating and individual reviews.

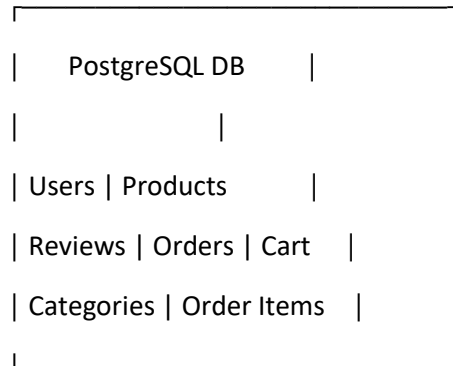
This feature provides social proof and helps potential buyers understand the experiences of other customers.

7. System Architecture Overview

The proposed application will follow a three-tier architecture.



| SQL Queries



The React frontend will communicate with the Node.js/Express.js backend through REST APIs. The Express.js backend will handle authentication, business logic, product operations, reviews, comparison functionality, cart management, and order processing. It will communicate with PostgreSQL for persistent data storage.

8. Scope Boundaries

To maintain a realistic academic project scope, the MVP will focus on the core features required for an operational Mobile Store.

Included in MVP

- User registration and login
- Product browsing
- Product search
- Product filtering
- Product details
- Mobile comparison

- Customer ratings and reviews
- Shopping cart
- Checkout
- Order creation
- Basic admin product/inventory management

Potential Future Enhancements

The following features may be considered after the MVP:

- AI-powered mobile recommendations
- Personalized product recommendations
- Wishlist
- Product notifications
- Advanced analytics
- Multiple payment gateways
- Delivery tracking
- Discount and coupon management
- Seller/vendor accounts
- Mobile application for Android/iOS

These future features are intentionally outside the initial MVP to keep the project feasible within the academic semester.

9. Conclusion

The first sprint establishes the foundation of the Mobile Store E-Commerce System before implementation begins. The project focuses on providing customers with a convenient platform for discovering, comparing, reviewing, and purchasing smartphones online.

The proposed architecture uses React.js for the frontend, Node.js/Express.js for the backend, and PostgreSQL for relational data management. The database design includes users, products, categories, reviews, orders, order items, carts, and cart items.

The combination of mobile comparison, customer reviews, product search, shopping cart, and online ordering provides the core functionality required for a practical and user-friendly mobile store while keeping the MVP achievable within the academic project timeline.